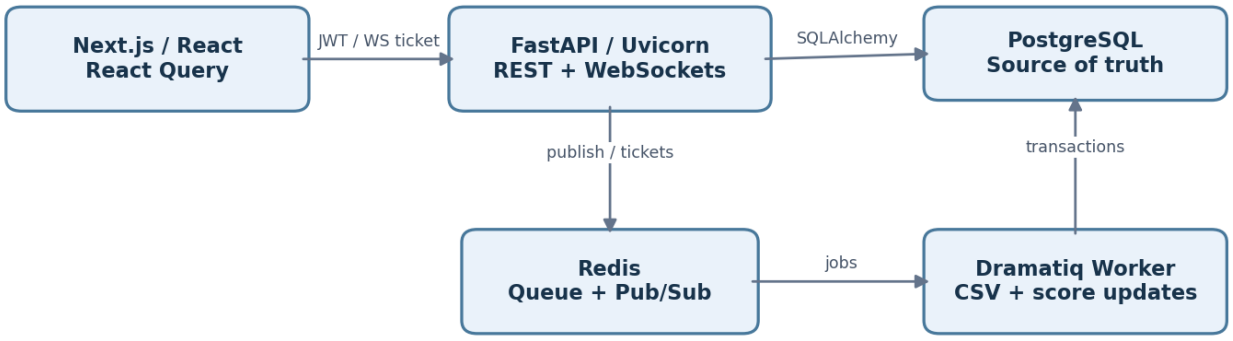
Engagement Pulse | Implementation Architecture

FastAPI implementation • November 2026 | Multi-tenant risk dashboard

- Consultants manage risks across assigned organisations; viewers access only their own organisation.
- PostgreSQL stores business state; Redis supports background jobs and real-time notifications.

1 Runtime architecture



Frontend	**Next.js 14, React 18, TypeScript, React Query; API proxy and WebSocket client.**
API	FastAPI on Uvicorn; JWT authentication, tenant checks, risk scoring, CSV job endpoints.
Persistence	PostgreSQL 16, SQLAlchemy 2, Alembic migrations; cached project risk score.
Async / events	Redis 7, Dramatiq worker, redis.asyncio Pub/Sub and single-use WS tickets.

Deployment

- Start all default services: docker compose up --build -d.
- Frontend: :3000; FastAPI: :8000; PostgreSQL, Redis and Dramatiq run as supporting services.
- API startup runs Alembic migrations and idempotent seeding.
- Django remains available under an optional Compose profile, not the default stack.

2 Data model & transactional rules

Organisation → Project → Risk	Project belongs to one organisation; risks belong to projects.
User / access mapping	Viewer belongs to one organisation; ConsultantOrganisationAccess maps consultants to multiple organisations.
CSVImportJob	Tracks project, creator, upload hash, status, row counts and errors.
Scoring configuration	SeverityWeight and StatusWeight stored in PostgreSQL; allowed statuses remain in application code.

Project score = min(100, Σ [severity weight × status weight]). Example: high/open = 3 × 3 = 9; closed = 0. Risk writes lock the project row and recalculate the cached score in the same database transaction.

3 Authentication & tenant boundary

- Access JWT stays in memory; refresh JWT uses an HttpOnly cookie via Next.js auth routes.
- FastAPI enforces organisation permissions on every protected operation.
- WebSocket access uses a 30-second, single-use Redis ticket and a separate tenant check.

4 CSV import — asynchronous execution



- Upload returns a job ID immediately; the frontend polls for status.
- Dramatiq validates/imports rows, recalculates the project score and records completion or failure.
- A shared Docker media volume stores uploads locally; object storage is a future improvement.

5 Live updates across instances



- Publish data_changed to fastapi:org:{id} after a successful commit.
- Each FastAPI instance receives Redis Pub/Sub events and forwards them to its own authorised WebSocket clients.
- React Query invalidates and refetches data; events are hints, not authoritative business records.

6 Key architecture decisions

FastAPI instead of Django	**Independent ASGI API using FastAPI + SQLAlchemy/Alembic; original Django implementation preserved for reference.**
Shared PostgreSQL, tenant checks	Simple operational model; server-side authorisation on APIs, jobs and WS. Per-tenant databases and RLS deferred.
Dramatiq + Redis	Background CSV processing without holding HTTP requests open; durable job status in PostgreSQL.
Redis Pub/Sub for WS	Cross-instance fan-out without polling; best-effort notifications, not guaranteed delivery.

7 Verification & boundaries

- Verified: one-command Docker startup, authentication/API proxy and CSV completion.
- Verified: live dashboard updates and cross-instance WebSocket delivery with tenant isolation.
- GitHub Actions backend and frontend jobs pass.
- Existing tests: 13 backend pytest and 34 frontend Jest; further edge-case tests remain planned.

8 Deliberately deferred — if time permitted

Browser E2E automation was intentionally deferred to prioritise secure tenant isolation, asynchronous CSV, cross-instance updates, Docker startup and CI.

Next, I would add a small Playwright smoke suite covering:

- Consultant login, organisation switching and risk edits reflected in the dashboard.
- Client viewer read-only access and inability to reach another organisation.
- CSV upload → job completion → updated project score.
- Two simultaneous sessions: consultant changes a risk and viewer sees the update without refresh.
- Expired session / refresh and logout behaviour.

Run a small Playwright smoke suite against isolated Docker Compose test data in CI; retain traces/screenshots on failures.

Other improvements if time permitted

- Strengthen backend integration and failure-path tests.
- Add audit logging and operational monitoring.
- Introduce managed secrets and object storage for uploads.
- Consider a transactional outbox if guaranteed event delivery becomes necessary.
- Avoid premature microservices; scale API and worker replicas independently first.